

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO1 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	G Navya Sri	Reg. No.:	192524170
Section / Batch:	B Tech-AI&DS	Date:	22-07-2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
GRAND TOTAL:		100 Marks	

SECTION A – CONCEPT MAPPING

Q1. Explain how time and space complexities interact. Extend the map by including performance and memory nodes. Justify why trade-offs are necessary. Interpret how this mapping supports balanced design decisions.

Q2. Explain how each case reflects different execution scenarios of an algorithm. Extend the map by linking each case to appropriate asymptotic bounds. Justify why average-case analysis is often more realistic. Interpret how this mapping helps in practical algorithm assessment.

Code of Conduct

I G Navya Sri (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G Navya Sri

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1. Time and Space Complexity Interaction

Explain

Time complexity measures how the number of operations grows with input size — the algorithm's performance. Space complexity measures how memory usage grows with input size. They are not independent metrics; most design decisions shift cost from one to the other rather than eliminating cost altogether.

- Divide and conquer and greedy methods are usually fast and lean on memory, since they do not store intermediate results.
- Dynamic programming speeds up execution precisely by spending extra memory — memoization tables trade space for time, turning exponential recursive calls into polynomial ones.
- Recursive algorithms pay a hidden space cost through the call stack, even when their time complexity looks efficient on paper.

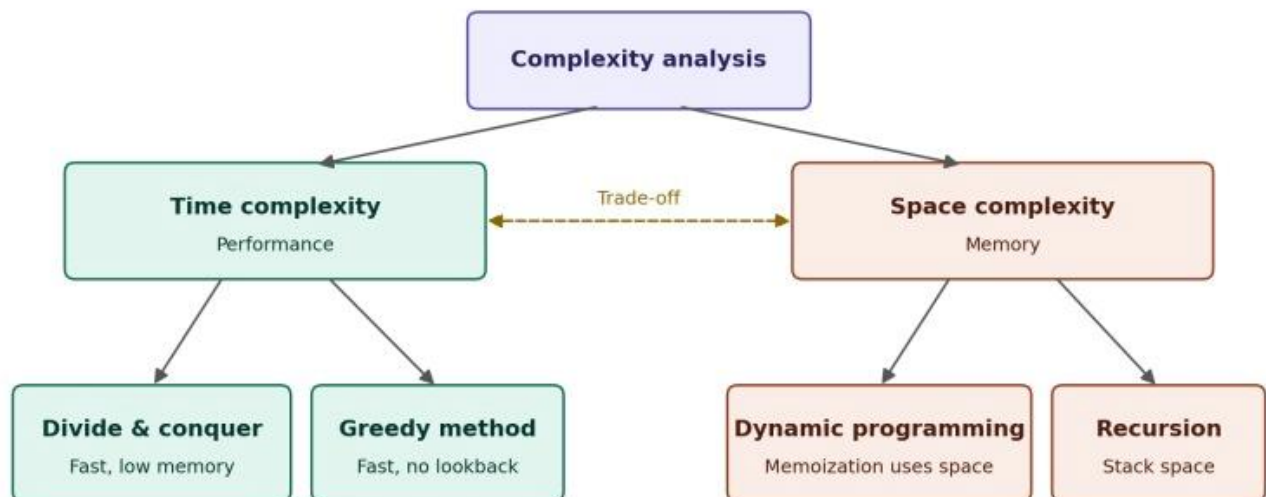


Figure 1: Extended concept map — complexity analysis branching into time (performance) and space (memory), linked by a trade-off relationship and grounded in DAA techniques.

Justify — why trade-offs are necessary

No algorithm can simultaneously minimize both time and space in every context, because reducing one usually means either recomputing (more time) or caching (more memory). Hardware constraints make this concrete: embedded or memory-limited systems must favor low space even at a time cost, while real-time systems such as trading platforms or games favor speed even if it consumes more RAM. Ignoring

this trade-off risks building something that is theoretically efficient on one metric but impractical in its actual deployment environment.

Interpret — how this supports balanced design

Placing time and space as siblings under a shared 'complexity analysis' node, connected by an explicit trade-off link, forces the same question for every technique underneath: what is being spent to gain what? That framing is what supports balanced design decisions — a designer scanning the map can choose divide-and-conquer for a memory-constrained device, or dynamic programming for a memory-rich, time-critical one, using the map as a decision aid rather than just a list of topics.

Q2. Case Analysis and Asymptotic Bounds

Explain

Each case reflects a different input arrangement for the same algorithm, not a different algorithm:

- Best case — the input arrangement requiring the fewest operations (e.g. an already-sorted array for insertion sort).
- Average case — performance expected over a random distribution of inputs.
- Worst case — the input arrangement forcing the maximum number of operations (e.g. a reverse-sorted array for insertion sort).

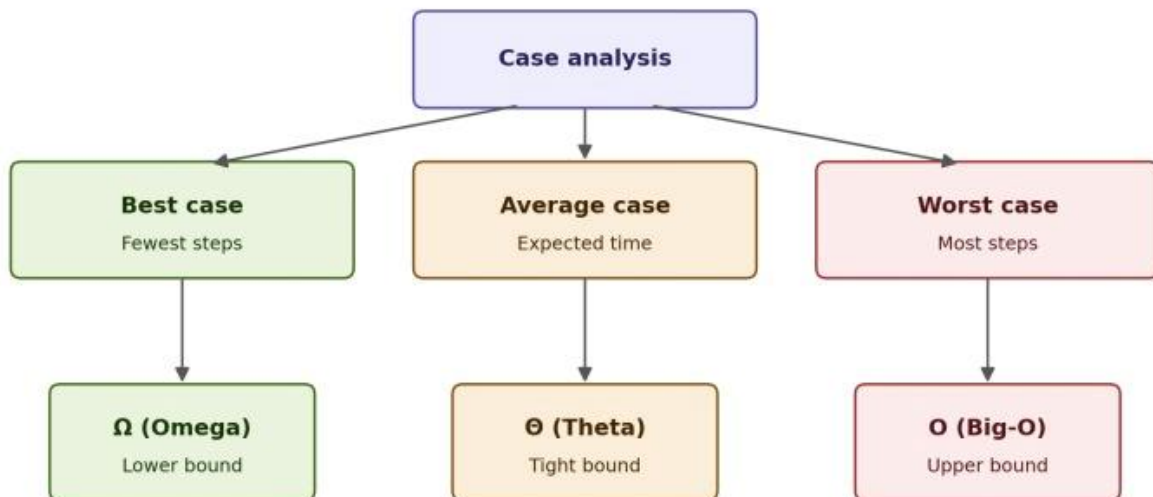


Figure 2: Extended concept map — case analysis branching into best, average, and worst case, each linked

to its matching asymptotic notation.

Extend — linking cases to asymptotic bounds

Best case corresponds to Ω (Omega) — a guaranteed lower bound on running time. Worst case corresponds to O (Big-O) — a guaranteed upper bound, the standard guarantee given in algorithm specifications. Average case corresponds to Θ (Theta) — a tight bound, since it captures growth that is neither the floor nor the ceiling but the realistic middle.

Justify — why average-case analysis is often more realistic

Worst-case inputs are frequently rare or adversarially constructed — a reverse-sorted array does not occur often in practice — and best-case inputs are equally uncommon. Real-world data usually falls somewhere in between, so average-case analysis, though harder to compute since it requires a probability distribution over inputs, better predicts how an algorithm will actually behave in production.

Interpret — how this helps practical algorithm assessment

Anchoring each case to its matching notation turns the map into a practical assessment tool: a developer can read Ω for the optimistic floor, Θ for the expected day-to-day experience, and O for the safety guarantee needed in worst-case-sensitive systems such as real-time or safety-critical software — using all three together rather than over-relying on a single number.